LAB ASSIGNMENT – 13

Task Description #1: Refactoring – Removing Code Duplication

Aim:

To use AI to identify and remove repeated code blocks in a Python program by creating a reusable function, while maintaining the same output.

Prompt:

Refactor the given Python code to remove duplication by creating a reusable function named rectangle_properties(length, width). Add a proper docstring, calculate the area and perimeter, maintain the same output, and verify the code using the given examples.

Refactored Python Code:

```
def rectangle_properties(length, width):
    """
    Calculate and display the area and perimeter of a rectangle.

    Parameters:
        length (int or float): Length of the rectangle.
        width (int or float): Width of the rectangle.
```

```
    Returns:
        tuple: Area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)

    return area, perimeter


# Function calls for different rectangles
rectangle_properties(5, 10)
rectangle_properties(7, 12)
rectangle_properties(10, 15)
```

Output:



Task Description #2: Refactoring – Optimizing Loops and Conditionals

Aim:

To optimize Python code by replacing nested loops with efficient set lookups while maintaining the same output and comparing execution time.

Prompt:

Optimize the given Python code by replacing nested loops with set lookups. Maintain the same logic and output, improve performance, and compare the execution time of the original and optimized code.

Refactored python code:

```
import time


# Given lists

names = ["Alice", "Bob", "Charlie", "David"]

search_names = ["Charlie", "Eve", "Bob"]
```

```
# Original code using nested loops
start = time.time()

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True

    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")

original_time = time.time() - start

# Optimized code using set lookup
names_set = set(names)

start = time.time()
```

```
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")


optimized_time = time.time() - start


# Performance comparison
print("\nPerformance Comparison:")
print("Original Execution Time:", original_time)
print("Optimized Execution Time:", optimized_time)
```

Output:



Task Description #3: Refactoring – Extracting Reusable Functions

Aim:

To refactor Python code by extracting repeated calculations into reusable functions, making the program modular, readable and easy to maintain.

Prompt:

Refactor the given Python code by creating a reusable function named calculate_total(price) to calculate tax (18%) and total price. Add a proper docstring and ensure the output remains the same.

Refactored Python Code:

```
def calculate_total(price):
    """
    Calculate the total price including 18% tax.

    Args:
        price (float): Original price of the product.

    Returns:
        float: Total price including tax.
    """
    tax = price * 0.18
    total = price + tax
```

```
    return total



# Calculate total price for different products

price1 = 250

print("Total Price:", calculate_total(price1))



price2 = 500

print("Total Price:", calculate_total(price2))
```

Output:

```
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> & 'c:\Program Files\Python314\python.exe' '"c:/Users/beebi/OneDrive/Desktop/ai assisted coding/lab13.3.py"'
Total Price: 295.0
Total Price: 590.0
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding>
```

Task Description #4: Refactoring – Replacing Hardcoded Values with Constants

Aim:

To identify and replace hardcoded values (magic numbers) with named constants to improve code readability, maintainability and reusability.

Prompt:

Refactor the given Python code by replacing hardcoded values with named constants such as PI and RADIUS. Define constants at the

beginning of the program and use them in all calculations while maintaining the same output.

Refactored Python Code:

```
# Define constants

PI = 3.14159

RADIUS = 7


# Calculate and display circle properties

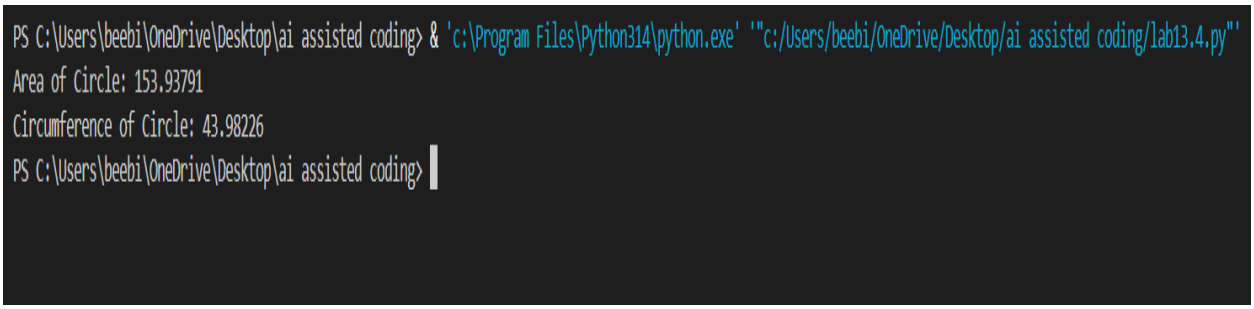
area = PI * (RADIUS ** 2)

circumference = 2 * PI * RADIUS


print("Area of Circle:", area)

print("Circumference of Circle:", circumference)
```

Output:



Task Description #5: Refactoring – Improving Variable Naming and Readability

Aim:

To improve Python code readability by replacing unclear variable names with meaningful names and adding explanatory comments without changing the program's functionality.

Prompt:

Refactor the given Python code by replacing unclear variable names with meaningful names, adding explanatory comments, and maintaining the same functionality and output.

Refactored Python Code:

```
# Define the base and height of the triangle

base = 10

height = 20

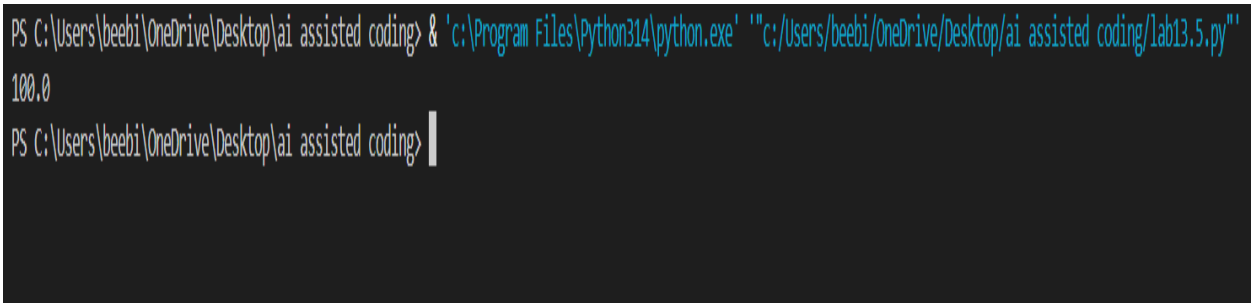

# Calculate the area of the triangle

area_of_triangle = (base * height) / 2


# Display the calculated area

print(area_of_triangle)
```

Output:



Task Description #6: Refactoring – Removing Redundant Conditional Logic

Aim:

To remove repeated if-else statements by creating a reusable function for calculating student grades while maintaining the original grading logic.

Prompt:

Refactor the given Python code by removing repeated if-else conditions and creating a reusable function named calculate_grade(marks). Add a proper docstring and maintain the same grading logic and output.

Refactored Python Code:

```
def calculate_grade(marks):
    """
    Calculate the grade based on student marks.

    Args:
        marks (int): Marks obtained by the student.
```

```
    Returns:
        str: Grade assigned to the student.
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"


# Calculate grades for students
marks = 85
print(calculate_grade(marks))

marks = 72
print(calculate_grade(marks))
```

Output:



Task Description #7: Refactoring – Converting Procedural Code to Functions

Aim:

To convert procedural Python code into modular functions by separating input, processing and output operations without changing the original behavior.

Prompt:

Refactor the given Python code by separating input, processing, and output into three reusable functions: get_input(), calculate_square(), and display_result(). Add docstrings and maintain the same output.

Refactored Python Code:

```
def get_input():
    """Get an integer input from the user."""
    num = int(input("Enter number: "))
    return num
```

```
def calculate_square(num):
    """Calculate and return the square of a number."""
    square = num * num
    return square


def display_result(square):
    """Display the calculated square."""
    print("Square:", square)


# Main program
number = get_input()
result = calculate_square(number)
display_result(result)
```

Output:

